Comprendre Minimax et `search.py`

Ce document explique Minimax et son moteur actuel dans search.py. Les sections 1 à 13 présentent les principes et des extraits pédagogiques du Minimax de base. Les sections 14 à 20 expliquent les nouveaux ajouts : délai, approfondissement itératif, mémoire des positions, statistiques et tests.

1. Le principe de Minimax

L'IA imagine ses coups et les réponses possibles de l'adversaire. Elle suppose que l'adversaire choisit lui aussi la meilleure réponse pour lui.

- **MAX** : tour de l'IA, qui conserve le plus grand score.
- **MIN** : tour de l'adversaire, qui conserve le plus petit score pour l'IA.

Tous les scores gardent le même point de vue : celui de l'IA.

```
                    IA : MAX
                   /        \
              Coup A        Coup B
                MIN            MIN
               /   \          /   \
              3     5        -2    8

Valeur de A : min(3, 5) = 3
Valeur de B : min(-2, 8) = -2
Choix de l'IA : A, car max(3, -2) = 3
```

L'IA ne choisit pas B en espérant obtenir 8 : l'adversaire peut lui imposer -2. Les nombres de cet exemple illustrent le raisonnement ; ce ne sont pas des mesures d'une position précise du plateau.

2. Les trois fonctions

Fonction	Question traitée	Résultat
`evaluate(board, ai_color)`	Quel est l'avantage actuel en pions ?	Un entier
`minimax(board, depth, current_color, ai_color)`	Quelle valeur obtient-on en anticipant les réponses ?	Un score
`choose_minimax_move(board, ai_color, depth=2)`	Quel coup faut-il jouer ?	`(ligne, colonne)` ou `None`

`Board` fournit les règles et les opérations sur le plateau. `MaybeMove` est un alias de type pour un coup éventuellement absent. Les annotations décrivent les paramètres et résultats ; elles ne convertissent pas automatiquement les valeurs.

3. L'évaluation : pourquoi soustraire les pions adverses ?

```
def evaluate(board, ai_color):
    my_discs = board.count(ai_color)
    opponent_discs = board.count(-ai_color)
    return my_discs - opponent_discs
```

Dans le projet, `BLACK = 1` et `WHITE = -1`. Ainsi, `-ai_color` désigne l'adversaire. Le signe moins s'applique à la **couleur**, pas au nombre de pions. On pourrait écrire `opponent_color = -ai_color` pour rendre cette étape explicite. La fonction reçoit une couleur, elle n'a pas besoin d'un objet joueur.

Pions de l'IA	Pions adverses	Évaluation
10	6	+4
5	9	-4
8	8	0

L'avantage calculé est bien le **score d'évaluation** récupéré par la recherche. Il mesure une avance ou un retard. Un score positif pendant la partie ne prouve pas une victoire : les pions peuvent encore être retournés. Une différence nulle pendant la partie ne prouve pas non plus un résultat final nul.

Cette stratégie est appelée évaluation absolue. Elle est simple, mais ne mesure ni les coins, ni la mobilité, ni la qualité stratégique des cases occupées.

4. Plateau réel et simulations

```
child = board.copy()
child.apply_move(move, current_color)
```

`child` est une copie indépendante. On y joue un coup pour répondre à la question « que se passerait-il si ce coup était joué ? ». Le plateau fourni reste intact. Chaque candidat repart du même plateau parent, pas du résultat du candidat précédent.

La vraie partie attend que l'IA choisisse son coup. Certaines branches simulées peuvent pourtant atteindre une fin de partie. `is_game_over()` vérifie simplement le plateau sur lequel on l'appelle : réel ou simulé.

Après la recherche, `choose_minimax_move()` retourne des coordonnées. C'est au moteur de jeu d'appliquer ce choix au plateau réel.

5. Profondeur et récursion

La récursion consiste à appeler `minimax()` depuis `minimax()` pour analyser la position suivante. Chaque appel possède ses propres variables locales. Il attend les résultats de ses appels enfants avant de calculer son minimum ou maximum.

Profondeur dans le code	Actions examinées, en l'absence de passe
0	Évaluer immédiatement le plateau
1	Un coup du joueur courant
2	Un coup du joueur courant puis une réponse adverse
3	Les deux actions précédentes puis un nouveau coup

Le code compte les **demi-coups**, c'est-à-dire les actions individuelles. La convention de la fiche de TP associe une profondeur de 1 à une action de chaque joueur : on peut convertir avec `depth_code = 2 * depth_tp`.

À chaque coup posé, on utilise `depth - 1`. Une passe ne consomme pas de profondeur dans cette implémentation. Une profondeur négative est refusée. Pour choisir un coup, `choose_minimax_move()` exige au moins 1 ; pour évaluer une position, `minimax()` accepte 0.

Une profondeur plus grande examine davantage de suites, mais coûte plus cher. Avec environ `b` choix par niveau et une profondeur `d`, le nombre de feuilles peut croître comme `b**d`. Commencer à 2 demi-coups permet de vérifier la logique avant d'augmenter la profondeur et de mesurer le temps.

6. Les paramètres de `minimax()`

Paramètre	Rôle
`board`	Position à analyser
`depth`	Nombre de coups restant à explorer
`current_color`	Joueur qui doit jouer dans cette position
`ai_color`	IA dont on évalue l'intérêt ; reste fixe

Si l'IA est blanche et que l'on simule une réponse noire, `current_color` vaut `BLACK`, mais `ai_color` reste `WHITE`. Un score positif reste favorable aux blancs.

7. Les conditions d'arrêt et les valeurs infinies

La première condition est la fin de partie : aucun des deux joueurs ne peut jouer.

```
if board.is_game_over():
    difference = evaluate(board, ai_color)
    if difference > 0:
        return float("inf")
    if difference < 0:
        return -float("inf")
    return 0
```

`is_game_over()` constate la fin. L'infini ne détecte pas la victoire et ne termine pas la vraie partie. Après ce constat, la différence de pions permet de savoir qui a gagné. On transmet alors une valeur numérique à l'appel parent pour comparer cette possibilité aux autres.

`float("inf")` construit l'infini positif ; `-float("inf")` est l'infini négatif. Ce n'est pas la même chose que `float(12)`, qui donne simplement `12.0`.

Possibilité simulée	Différence de pions	Valeur retournée
Partie terminée, victoire	+2	+∞
Partie en cours à la limite de recherche	+12	12
Partie terminée, défaite	-2	-∞
Partie terminée, égalité	0	0

Sans cette distinction, MAX préférerait l'avance provisoire de 12 à la victoire certaine par 2 pions. Avec +∞, la victoire est prioritaire. Inversement, -∞ classe une défaite certaine sous toute estimation finie.

Ce choix est une convention d'évaluation, pas une obligation de Minimax. Avec la seule différence de pions sur 64 cases, on pourrait utiliser `1000` et `-1000`. Si l'évaluation change, il faudrait vérifier que ces nombres dominent toujours tous les scores intermédiaires. Les infinis évitent de choisir cette borne.

Toutes les victoires ont ici la même valeur : l'IA ne préfère pas spécialement une victoire par 20 pions à une victoire par 2 pions. L'évaluation limitée en profondeur peut aussi se tromper sur les branches qui ne sont pas terminées.

Un objet `Victory()` serait possible, mais il faudrait définir comment le comparer aux autres résultats. Les nombres fonctionnent directement avec `min()` et `max()`.

La deuxième condition d'arrêt intervient seulement si la partie continue :

```
if depth == 0:
    return evaluate(board, ai_color)
```

On retourne alors une estimation. Vérifier la fin avant la profondeur garantit qu'une fin simulée atteinte exactement à profondeur 0 reçoit un résultat définitif.

8. Les coups légaux et la passe

```
moves = board.get_legal_moves(current_color)
if not moves:
    return minimax(board, depth, -current_color, ai_color)
```

`if not moves` signifie que la liste est vide. Le joueur passe : même plateau, même profondeur, autre joueur. Le point de vue `ai_color` ne change pas. Si les deux joueurs étaient bloqués, le test de fin aurait déjà arrêté la recherche.

9. La branche MAX

```
best_score = -float("inf")
for move in moves:
    child = board.copy()
    child.apply_move(move, current_color)
    score = minimax(child, depth - 1, -current_color, ai_color)
    best_score = max(best_score, score)
return best_score
```

Cette branche est utilisée si `current_color == ai_color`.

1. Initialiser la meilleure valeur au plus bas possible.
2. Copier le plateau pour chaque coup légal.
3. Jouer ce coup sur la copie.
4. Explorer la réponse adverse avec un niveau de moins.
5. Garder le plus grand score.
6. Retourner cette valeur à l'appel précédent.

Pour des scores successifs `3`, `-2`, `5`, la meilleure valeur passe de `-∞` à `3`, reste `3`, puis devient `5`.

Ici, l'infini est une initialisation, pas une défaite constatée. Commencer à 0 serait incorrect si tous les coups ont des valeurs négatives : on risquerait de conserver un score de 0 qu'aucun coup ne permet d'obtenir.

10. La branche MIN

Le fonctionnement est identique, mais l'adversaire minimise le score de l'IA :

```
best_score = float("inf")
# Pour chaque coup : copier, jouer, appeler minimax...
best_score = min(best_score, score)
```

Pour `3`, `-2`, `5`, le résultat est `-2`. On commence à `+∞` pour pouvoir conserver le plus petit résultat, même si tous les scores sont positifs. Cette initialisation ne signifie pas qu'une victoire a déjà été découverte.

11. Choisir les coordonnées du meilleur coup

`minimax()` retourne une valeur. `choose_minimax_move()` associe chaque valeur au coup initial qui la produit.

```
best_move = moves[0]
best_score = -float("inf")
```

Le premier coup légal sert de choix initial. Cela garantit un résultat légal même si tous les scores valent `-∞` : aucun ne serait strictement supérieur à `best_score`, mais `best_move` contient déjà un coup.

Pour chaque candidat, la fonction copie le plateau, joue le coup et appelle :

```
score = minimax(child, depth - 1, -ai_color, ai_color)
```

Le premier coup est déjà simulé : il consomme un niveau. C'est maintenant à l'adversaire de jouer. Si le score est meilleur, on mémorise les deux informations :

```
if score > best_score:
    best_score = score
    best_move = move
```

Le test strict `>` conserve le premier coup rencontré en cas d'égalité. La fonction retourne `best_move`, ou `None` si aucun coup n'est légal.

12. Exemple d'utilisation et déroulement complet

```
from board import Board
from constants import WHITE
from search import choose_minimax_move

board = Board()
move = choose_minimax_move(board, WHITE, depth=2)
print(move)  # Coordonnées choisies ; le plateau est encore inchangé.
```

Pour chaque coup blanc, la recherche examine les réponses noires, évalue les positions obtenues et garde leur minimum. Elle compare ensuite les valeurs des coups blancs et choisit le maximum. Ce calcul correspond à « choisir la meilleure option en anticipant la réponse la plus défavorable de l'adversaire ».

13. Vérifications utiles et suites possibles

- Vérifier que le plateau original reste intact après une recherche.
- À profondeur 1, vérifier que le coup choisi a la meilleure valeur immédiate, en tenant compte de la priorité des résultats terminaux.
- Vérifier les passes, les victoires, les défaites et les égalités.
- Vérifier qu'un coup légal est retourné même si tous les coups perdent.
- Vérifier le raisonnement MAX/MIN sur un petit arbre connu.

Les prochaines évolutions possibles sont Alpha-Bêta et les évaluations positionnelle, mobilité et mixte. La gestion du temps et la mémoire sont maintenant implémentées et expliquées dans les sections suivantes.

14. Du Minimax de base au moteur complet

Les sections précédentes décrivent le raisonnement MAX/MIN. Le code actuel conserve ce raisonnement et ajoute la gestion du temps, la réutilisation des positions et des mesures pour le TP. Il ne contient toujours pas de coupures Alpha-Bêta.

Avant	Maintenant
Profondeur fixe seulement	Profondeur maximale et délai optionnel
Recherche recalculée à chaque appel	Mémoire persistante propre à chaque IA
Un coup retourné	Coup, score et statistiques disponibles
Pas de gestion d'interruption	Dernière itération complète ou coup de secours

Le critère reste la différence du nombre de pions. Le score conserve le point de vue de l'IA : MAX cherche sa plus grande valeur, MIN sa plus petite. Les copies de plateau protègent toujours la partie réelle.

Les nouveaux objets

- `MinimaxEngine` : organise la recherche et garde la mémoire entre les tours.
- `_SearchContext` : transmet l'échéance, la table et les compteurs aux appels récursifs. Le préfixe « _ » indique un élément interne.
- `SearchTimeout` : exception utilisée pour interrompre la recherche en cours.
- `SearchStats` : regroupe les mesures d'une décision.
- `SearchResult` : regroupe le coup, sa valeur et ses mesures.

`@dataclass` construit automatiquement l'initialisation des objets de données à partir de leurs champs. Cela évite d'écrire manuellement plusieurs affectations dans un constructeur.

Comment les fonctions se relient

```
AIPlayer.choose_move(board)
    → son moteur.search(board, couleur, profondeur, délai)
        → minimax(...) pour chaque coup candidat
            → autres appels minimax(...) sur des copies
        ← SearchResult(coup, score, statistiques)
    ← coordonnées du coup, destinées à Game
```

`choose_minimax_move()` reste une fonction pratique qui retourne seulement les coordonnées. Elle délègue désormais au moteur : la boucle de sélection décrite en section 11 se trouve dans `MinimaxEngine.search()`.

15. La limite de temps

```
started = perf_counter()
deadline = started + time_limit
```

`perf_counter()` mesure des durées indépendamment des changements de l'heure civile. Si le départ vaut 100 et le budget 2 secondes, l'échéance vaut 102. Le programme contrôle régulièrement si cette échéance est atteinte :

```
if self.deadline is not None and perf_counter() >= self.deadline:
    raise SearchTimeout
```

`None` signifie qu'aucun délai n'a été demandé. L'exception `SearchTimeout` remonte les appels récursifs jusqu'au `except` du moteur. Elle interrompt la réflexion, pas la partie ni le programme.

Le délai est coopératif. Une méthode de `Board` déjà engagée finit avant le prochain contrôle. Un léger dépassement reste donc possible. Le moteur calcule aussi les coups légaux initiaux pour toujours disposer d'un secours, même avec un budget nul.

16. L'approfondissement itératif

```
depths = [depth] if time_limit is None else range(1, depth + 1)
```

Sans délai, le moteur cherche directement à la profondeur demandée. Avec un délai, il essaie 1, puis 2, puis 3 demi-coups, jusqu'au maximum. Cette progression permet d'avoir un choix complet disponible avant de tenter une recherche plus coûteuse.

Étape	Résultat conservé
Niveau 1 terminé	Meilleur coup au niveau 1
Niveau 2 terminé	Meilleur coup au niveau 2
Niveau 3 interrompu	Le résultat complet du niveau 2

Les variables `candidate_move` et `candidate_score` décrivent l'itération en cours. Les variables `best_move` et `best_score` gardent le dernier résultat complet. On remplace les secondes seulement après avoir examiné tous les coups du niveau. Sinon, certains coups seraient comparés à une profondeur différente des autres.

Si aucun niveau n'a terminé, le premier coup légal sert de secours. On annonce alors `completed_depth=0`, `score=None` et `timed_out=True`. **None n'est pas un score nul : la qualité du secours n'a pas été établie.**

La profondeur reste comptée en actions individuelles : `depth=6` correspond à un maximum de six demi-coups, soit trois selon la convention du TP. Une passe ne diminue pas cette profondeur.

17. Mémoriser les positions déjà traitées

La table de transposition est un dictionnaire. Elle associe un problème de recherche à sa valeur déjà calculée :

```
key = (board.size, board.to_key(), current_color, ai_color, depth)
if key in context.table:
    context.stats.cache_hits += 1
    return context.table[key]
```

Élément de clé	Pourquoi il est nécessaire
Taille et grille	Identifier le plateau et la disposition des pions
current_color	La même grille peut avoir un autre joueur au trait
ai_color	Le score dépend du point de vue adopté
depth	Deux horizons de recherche peuvent donner deux valeurs différentes

`board.to_key()` représente la grille sous forme d'octets utilisables comme clé. Le dictionnaire compare les clés, pas uniquement une empreinte approximative.

On réutilise uniquement une valeur avec la même profondeur restante. Une valeur à profondeur 4 ne remplace pas arbitrairement celle à profondeur 2. L'évaluation est actuellement toujours absolue ; si elle change, il faudra vider la mémoire ou inclure l'identité de l'évaluation dans la clé.

Pourquoi enregistrer seulement les calculs terminés ?

`context.save(key, score)` est appelé lorsqu'un nœud a une valeur complète. Si un enfant déclenche une interruption, son parent n'atteint pas cette instruction. Les enfants déjà terminés peuvent rester mémorisés, mais pas une valeur partielle du parent.

« Valeur exacte » signifie ici exacte pour Minimax à la profondeur et avec l'évaluation choisies. Cela ne signifie pas qu'une position intermédiaire a été résolue jusqu'à la fin de la partie.

Limiter la consommation mémoire

La capacité par défaut est de 100 000 entrées. Quand la table est pleine, le moteur retire la plus ancienne entrée avant d'ajouter une nouvelle clé. C'est une politique simple fondée sur l'ordre d'insertion, pas sur la date du dernier accès. Une suppression peut demander de refaire un calcul, mais ne change pas sa valeur.

18. Réutiliser l'analyse au tour suivant

Chaque `AIPlayer` crée son propre `MinimaxEngine` dans son constructeur. Ce moteur reste vivant entre les appels à `choose_move()`. Deux IA possèdent donc des dictionnaires distincts.

```
self.engine = MinimaxEngine()

# À chaque tour :
self.last_result = self.engine.search(
    board, self.color, self.depth, self.time_limit
)
return self.last_result.move
```

Exemple : une recherche a exploré quatre demi-coups. Après deux coups réels, certains sous-arbres du nouveau plateau ont déjà été étudiés avec deux demi-coups restants. Le moteur peut les retrouver. Le gain dépend des positions, des profondeurs et des entrées encore en mémoire ; il n'est pas garanti à chaque appel.

On conserve ainsi les valeurs d'un ensemble de positions et de sous-arbres, sans construire un arbre explicite avec des liens parents/enfants. `engine.clear()` vide la table et le dernier résultat du moteur pour commencer une expérience avec une mémoire vide.

19. Lire le résultat et les statistiques

```
result = engine.search(board, WHITE, depth=6, time_limit=2.0)
print(result.move)
print(result.score)
print(result.stats)
```

Champ	Signification
move	Coup choisi ; None si aucun coup n'est légal
score	Valeur du dernier niveau terminé ; None pour un secours non évalué ou une absence de coup
stats.nodes	Appels récursifs visités, accès au cache compris
stats.cache_hits	Résultats retrouvés dans la mémoire
stats.completed_depth	Dernière profondeur entièrement terminée
stats.elapsed_seconds	Durée totale, préparation incluse
stats.timed_out	Une recherche a été interrompue par le délai

Les compteurs cumulent les itérations, y compris le travail du niveau interrompu. `nodes` ne compte pas les positions uniques ni les racines des itérations : il compte les entrées dans la fonction récursive après son contrôle initial du temps.

Pour comparer des performances, préciser si le cache était vide ou déjà alimenté. Une deuxième recherche identique peut être plus rapide grâce à la première.

Configurer un joueur

```
from player import AIPlayer
from constants import WHITE

white = AIPlayer(WHITE, depth=6, time_limit=2.0, name="IA blanche")
move = white.choose_move(board)
print(white.last_result.stats)
```

L'ancien appel `choose_minimax_move(board, color, depth)` reste possible. Il crée un moteur temporaire si aucun n'est fourni. Pour conserver la mémoire entre appels, utiliser le même `engine` en argument ou passer par un `AIPlayer`.

20. Comprendre les dix tests

Le fichier `test_search.py` utilise `unittest`. Chaque méthode commençant par `test_` vérifie un comportement. Une assertion compare une observation au résultat attendu ; une assertion fausse fait échouer le test.

La fonction `reference()` est un Minimax simple sans mémoire ni délai. Elle sert à comparer les résultats à faible profondeur. Plusieurs tests utilisent des plateaux 4×4 pour garder les calculs exhaustifs rapides. Ces tests ciblés ne constituent pas une preuve pour toutes les positions possibles.

1. `test_matches_reference_and_preserves_board` : compare les scores et les coups au calcul de référence pour les deux couleurs et plusieurs profondeurs. Vérifie que la grille d'origine ne change pas.
2. `test_terminal_results_at_depth_zero` : vérifie victoire, défaite et égalité sur des plateaux terminés, même à profondeur zéro. Aucun coup ne doit être proposé sur un plateau plein.
3. `test_pass_and_forced_loss` : vérifie qu'un joueur bloqué passe alors que son adversaire peut jouer. Vérifie aussi qu'un coup légal est retourné même s'il mène à une défaite certaine.
4. `test_cache_reuse_and_separation_of_depth_and_perspective` : répète une recherche et attend le même résultat avec moins de nœuds. Compare ensuite les recherches d'autres profondeurs et couleurs à la référence.
5. `test_reuse_on_a_later_turn` : joue deux coups après une recherche profonde, puis vérifie que le moteur retrouve des analyses utiles et reste équivalent à un moteur neuf.
6. `test_zero_budget_returns_legal_fallback` : avec un budget nul, exige un coup légal de secours, aucun score prétendument calculé, une profondeur terminée de zéro et un signal d'interruption.
7. `test_timeout_keeps_last_complete_iteration` : fait expirer une horloge simulée au début du niveau 2. Exige de conserver le résultat du niveau 1, de ne pas mémoriser de parent incomplet et de pouvoir reprendre correctement.
8. `test_iterative_deepening_reaches_requested_depth` : fige l'horloge pour vérifier que le moteur atteint le niveau 3 sans interruption et obtient la valeur attendue.
9. `test_memory_limit_and_independent_players` : limite la table à deux entrées pour forcer l'éviction, vérifie le résultat, l'indépendance des mémoires de deux IA et la remise à zéro du moteur.

10. `test_invalid_parameters` : vérifie que des profondeurs invalides, des délais négatifs ou non finis et une capacité nulle sont refusés.

`patch()` remplace temporairement l'horloge ou un appel pendant un test. Cela permet de provoquer exactement le scénario voulu, sans attendre réellement et sans dépendre de la rapidité du PC. Les tests d'horloge simulée vérifient le comportement d'interruption, pas une garantie de temps réel strict.

```
python -m unittest test_search -v
```

Cette commande nécessite un interpréteur Python disposant de NumPy. Les dix tests ont été exécutés avec succès lors de cette mise à jour.